35 Testing applications with xUnit

This chapter covers

- Testing in ASP.NET Core
- Creating unit test projects with xUnit
- Creating Fact and Theory tests

When I started programming, I didn't understand the benefits of automated testing. It involved writing so much more code. Wouldn't it be more productive to be working on new features instead? It was only when my projects started getting bigger that I appreciated the advantages. Instead of having to run my app and test each scenario manually, I could click Play on a suite of tests and have my code tested for me automatically.

Testing is universally accepted as good practice, but how it fits into your development process can often turn into a religious debate. How many tests do you need? Should you write tests before, during, or after the main code? Is anything less than 100 percent coverage of your code base adequate? What about 80 percent?

This chapter won't address any of those questions. Instead, I focus on the mechanics of creating a test project in .NET. In this chapter I show you how to use isolated unit tests to verify the behavior of your services in isolation. In chapter 36 we build on these basics to create unit tests for an ASP.NET Core application, as well as create integration tests that exercise multiple components of your application at the same time.

TIP For a broader discussion of testing, or if you're brand-new to unit testing, see *The Art of Unit Testing*, 3rd ed., by Roy Osherove (Manning, 2024). If you want to explore unit test best practices using C# examples, see *Unit Testing*